Adaptability questions

Design Gurus

Here are a few behavioral questions related to technical competence and quality:

Intention of the question: The interviewer wants to see you handle a problem where the usual tools do not help. They are also checking what you do when the cause turns out to sit outside your own code.

Pitfalls to avoid: Avoid a problem that was only hard because you were new. Do not tell a story where the resolution is that somebody else fixed it. If the cause was in a dependency, say what you did while you waited.

Sample Answer 1:

Situation: "Our uploads started failing for files over about 8 megabytes, but only for customers in one region, and only through our mobile app. Our code had not changed in six weeks. The same file uploaded fine from a browser."

Task: "I owned the upload service. Every signal said the problem was not in my code, which is the least useful conclusion available, because the customers were still failing. I had to find a cause I could not change."

Action: "I worked outward from what varied. The three variables were region, client, and file size, so I tested them one at a time instead of together. That took a day and gave a clear result. It failed only where all three met. Then I captured the raw request from a failing device. The mobile library was uploading in chunks. Our regional proxy was buffering the whole body before forwarding it, with a limit just under 8 megabytes. Neither piece was ours and I could not patch either. So I changed what I did control. I had the service advertise a smaller chunk size to mobile clients, which kept every request under the limit."

Result: "Uploads in that region went from about 60 percent failure on large files to under 1 percent, within two days of finding it. The vendor raised the proxy limit about five weeks later and I removed my workaround a month after that. Working outward from what varied is the step I would repeat. Three variables meant eight combinations, and eight combinations is a day of dull work that replaced five weeks of theories."

Sample Answer 2: "A different one was harder, because nothing varied at all. A nightly report was wrong by a few cents about once a month, with no pattern in the accounts or the amounts. I got nowhere for two weeks. What solved it was giving up on the code and reading the data instead. I found nine rows out of four million with a currency code in lower case. They were written by an importer retired two years earlier. The bug was not in any running code. It was in data no current system could have produced. When a problem has no pattern in behavior, look for a pattern in the history of the data."

Follow-up you should expect: "You said the cause was in a vendor's proxy. How did you get it fixed?"

Why they ask: They want to know whether you can move something you do not own. The alternative is stopping at a workaround and calling it finished.

How to answer it: "I sent the vendor the packet capture and an exact reproduction, not a description. That mattered. Their first reply was that the limit is configurable and ours was set correctly, which was true and not the problem. A capture showing the request failing at 7.9 megabytes moved it to their engineering team in one exchange. I also gave our account manager the number of affected customers each week, so the pressure was not only mine. Five weeks is slow, but with a description rather than a capture, I do not think it would have moved at all."

Question 2: What is your approach to balancing the need for high-quality code with the need to deliver features quickly in a fast-paced environment?

Intention of the question: The interviewer wants to know where you actually draw the line, not that you value both. They are listening for whether you decide this case by case or apply one rule everywhere.

Pitfalls to avoid: Avoid saying you never compromise on quality, which is not credible on a fast-moving team. Avoid saying you ship first and clean up later, because the cleanup rarely happens. Give the rule you use, and an example of applying it in each direction.

Sample Answer 1:

Situation: "We were launching a feature to a single pilot customer in three weeks. The same code was expected to serve all customers about six months later. The team disagreed about how carefully to build it."

Task: "As the senior engineer I had to set the standard. Building it as though it were general would miss the pilot date. Building it only for the pilot would mean rewriting it under load later."

Action: "I split the code by how expensive it would be to change later, not by how important it felt. The data model and the public API went in slowly, because other teams and stored records depend on those. Everything behind them was the whole processing path. We wrote that quickly and accepted it would be replaced. I wrote that split into three lines of the design document. That made it a decision rather than an accident. I also required tests on the interfaces only, not the internals. That way we could rewrite the internals without rewriting the tests."

Result: "We hit the pilot date. Six months later we replaced about 70 percent of the processing code. That took five weeks and changed no stored data and no caller. The data model has not changed since. The rule I use now is that quality effort should follow the cost of changing something later. Code that is cheap to replace does not deserve the same care as a schema that is expensive to replace."

Sample Answer 2: "I have also called it the other way and been wrong. Under a deadline I skipped input validation on an internal admin tool. The only users were four people on our own team. Eight months later that tool was open to about 200 support agents. One of them corrupted 3,000 records with a badly formed paste. Nobody ever decided to widen the audience. It grew one person at a time. I now ask who could plausibly use this in a year, not who uses it today. An internal tool with four users is the most likely thing in the codebase to end up with two hundred."

Follow-up you should expect: "Who decides which category a piece of code falls into?"

Why they ask: The rule sounds clean in a story. They want to know whether it survives contact with a team that disagrees.

How to answer it: "I write the split into the design document, and anybody can argue with it there, before the code exists. Twice somebody has moved an item from the fast side to the careful side. Both times they were right. What I do not allow is that argument happening in code review three weeks later, one file at a time. That is the same disagreement at ten times the cost. If we cannot agree in the document, we escalate then, not later."